Professor Cubazoid's 3D Tetris

Technical Appendix

STA 561 Final Project

ZRTMRH Nicholas Popescu Isabelle Dorage Rally Lin

Contents

1 Problem Formulation **2**

2 System Architecture Overview **2**

3 Polycube Representation **2**
- 3.1 Normalization . . . 3
- 3.2 Rotation Group . . . 3
- 3.3 Placement Enumeration . . . 3

4 Phase 1: Algorithm X with Dancing Links **4**
- 4.1 Exact Cover Formulation . . . 4
- 4.2 Dancing Links (DLX) . . . 4
- 4.3 Performance . . . 5

5 Phase 2: CuboidNet Neural Network **5**
- 5.1 Architecture . . . 5
- 5.2 State Encoding . . . 6
- 5.3 Training . . . 6

6 NN-Guided Beam Search **7**
- 6.1 Algorithm . . . 7
- 6.2 Key Heuristics . . . 7
- 6.3 Frontier Search . . . 7

7 Autodidactic Iteration (ADI) **7**

8 Hybrid Solver and Size-Gated Routing **8**
- 8.1 `hybrid_solve()` Pipeline . . . 8
- 8.2 `solve_size_gated()` Routing . . . 8

9 Block Decomposition for Large Grids **8**
- 9.1 Strategy . . . 8
- 9.2 Block Size Selection . . . 9

10 Rectangular Sub-Box Decomposition **9**

10.1 Motivation . . . 9
10.2 Algorithm . . . 9
10.3 Piece Allocation . . . 9
10.4 Performance by Decomposition Method . . . 10

11 Timeout and Safety Mechanisms **10**
11.1 Wall-Clock Timeout (SIGUSR1 + Threading) . . . 10
11.2 Per-Block DLX Timeout . . . 10
11.3 Multiprocessing Isolation . . . 11

12 Solution Verification **11**

13 Test Methodology and Results **11**
13.1 Test Infrastructure . . . 11
13.2 Fault Test Strategies . . . 11
13.3 Constructive Test Generation . . . 12
13.4 Results: $N = 3$ through $N = 14$ (1,080 total tests) . . . 12
13.5 Results: $N = 14$ through $N = 24$ . . . 13
13.6 Combined Results . . . 13

14 Visualization **13**
14.1 Static 3D Plot . . . 13
14.2 Animated Assembly . . . 13

15 Code Structure and Reproduction **13**
15.1 Repository Structure . . . 13
15.2 Reproduction Instructions . . . 14
15.3 Dependencies . . . 15

16 References **15**

1 Problem Formulation

Input: A list of k polycube pieces $P_1, P_2, \ldots, P_k$, where each piece P_i is a set of 3D integer coordinates representing connected unit cubes. Each piece has size $|P_i| \in \{3, 4, 5\}$.

Output: Either a placement function $f : \{1, \ldots, k\} \to \mathcal{T}$ mapping each piece to a rigid transformation (rotation + translation) such that:

- $\bigcup_{i=1}^{k} f(P_i) = \{0, \ldots, N-1\}^3$ for some integer N (the pieces perfectly tile an $N \times N \times N$ cube), and
- $f(P_i) \cap f(P_j) = \emptyset$ for all $i \neq j$ (no overlaps),

or `None` if no such placement exists.

Necessary condition: $\sum_{i=1}^{k} |P_i| = N^3$ for some positive integer N.

This is an instance of the **exact cover** problem, which is NP-complete in general. Our approach combines exact algorithms (tractable for small instances), learned heuristics (fast but incomplete), and divide-and-conquer decomposition (enabling scaling to large instances).

2 System Architecture Overview

The solver is organized as a three-layer pipeline, controlled by a size-gated orchestrator:

```
Input: pieces, grid_size N
         |
         v
  solve_size_gated()  (hybrid_solver.py)
         |
    +----+-----------------------------+
    |                                  |
    v  N <= 9              N >= 10     v
  DLX exact       Block Decomposition
  (exact cover)   +-- blockwise_Xcube
                  +-- rect_blockwise
  N <= 4: 30s     +-- general blocks
  N 5-9: up to          |
   600s budget      if fails
                         v
                  hybrid_solve()
                  NN beam -> retry ->
                  frontier -> DLX
```

3 Polycube Representation

File: `phase1/polycube.py`

Each polycube piece is represented as a `frozenset` of `(x, y, z)` integer tuples.

3.1 Normalization

A piece is **normalized** by translating its bounding box to the origin:

```
def normalize(coords):
    min_x = min(c[0] for c in coords)
    min_y = min(c[1] for c in coords)
    min_z = min(c[2] for c in coords)
    return frozenset((x-min_x, y-min_y, z-min_z)
                     for x, y, z in coords)
```

3.2 Rotation Group

The 24 orientation-preserving symmetries of the cube (rotation group O) are precomputed as coordinate transformations. Each rotation is a function $(x, y, z) \mapsto (\pm x_{\sigma(1)}, \pm x_{\sigma(2)}, \pm x_{\sigma(3)})$ for some axis permutation σ and sign pattern with positive determinant.

```
ROTATIONS = [...]  # 24 rotation functions

def get_orientations(piece):
    """Return all unique normalized rotations."""
    seen = set()
    result = []
    for rot in ROTATIONS:
        rotated = normalize(frozenset(rot(*c) for c in piece))
        if rotated not in seen:
            seen.add(rotated)
            result.append(rotated)
    return result
```

For a piece with no internal symmetry, this produces 24 distinct orientations. Symmetric pieces produce fewer (e.g., a $1 \times 1 \times 3$ rod has only 3 distinct orientations).

3.3 Placement Enumeration

For each orientation of a piece within an $N \times N \times N$ grid, we enumerate all valid translations:

```
def get_all_placements(piece, grid_size):
    placements = []
    for orientation in get_orientations(piece):
        bx = max(c[0] for c in orientation)
        by = max(c[1] for c in orientation)
        bz = max(c[2] for c in orientation)
        for dx in range(grid_size - bx):
            for dy in range(grid_size - by):
                for dz in range(grid_size - bz):
                    placed = frozenset(
                        (x+dx, y+dy, z+dz)
                        for x,y,z in orientation)
                    placements.append(placed)
    return placements
```

A rectangular variant `get_all_placements_rect(piece, gx, gy, gz)` handles non-cubic sub-boxes.

4 Phase 1: Algorithm X with Dancing Links

Files: `phase1/dlx_solver.py`, `phase1/solver.py`

4.1 Exact Cover Formulation

The polycube packing problem is formulated as an **exact cover** problem:

- **Primary columns:** One column per grid cell $(x, y, z) \in \{0, \ldots, N-1\}^3$, plus one column per piece index $i \in \{1, \ldots, k\}$. Each cell must be covered exactly once; each piece must be used exactly once.
- **Rows:** For each piece i and each valid placement p of piece i, create a row that covers the piece-i column and all cell columns $(x, y, z) \in p$.

A solution is a subset of rows that covers every primary column exactly once—i.e., a valid tiling.

4.2 Dancing Links (DLX)

The exact cover matrix is represented as a toroidal doubly-linked list (Knuth, 2000). Each node has four pointers (up, down, left, right) and a column reference:

```
class Node:
    __slots__ = ('left','right','up','down','column','row_id')

class Column(Node):
    __slots__ = ('size', 'name')  # size = number of 1s

class DLX:
    def solve(self, find_all=False):
        # Algorithm X with column-selection heuristic:
        # choose column with minimum size (MRV)
```

Algorithm X proceeds recursively:

1. If no primary columns remain, a solution is found.
2. Choose the primary column c with the fewest rows (minimum remaining values heuristic).
3. For each row r covering column c:
   (a) Include r in the partial solution.
   (b) For each column j covered by r, remove column j and all rows intersecting j ("cover" operation—implemented by pointer surgery on the linked list).
   (c) Recurse.
   (d) Undo step (b) ("uncover"—restore pointers in reverse order).
4. If no row covers c, backtrack.

The MRV heuristic is critical: by choosing the most constrained column first, the algorithm prunes dead branches early.

4.3 Performance

Grid Size	Volume	Pieces	Placements	DLX Time
3^3	27	7	∼2,000	< 0.1s
4^3	64	∼16	∼15,000	1–5s
5^3	125	∼30	∼80,000	10–60s
6^3	216	∼50	∼250,000	30–120s
7^3	343	∼80	∼600,000	20–70s
8^3	512	∼120	∼1,200,000	30–90s
9^3	729	∼170	∼2,500,000	60–300s

Table 1: DLX solver performance by grid size.

Beyond $9 \times 9 \times 9$, direct DLX becomes impractical, motivating the block decomposition approach.

5 Phase 2: CuboidNet Neural Network

File: `phase2/nn_model.py`

5.1 Architecture

CuboidNet is a 3D residual convolutional neural network, inspired by DeepCube (Agostinelli et al., 2019) and AlphaGo (Silver et al., 2016).

Input tensor: $(B, 1{+}M, N, N, N)$ where B is batch size, M is max pieces, N is grid size.

- Channel 0: binary occupancy grid (1 = cell occupied, 0 = empty)
- Channels 1 through M: per-piece indicator volumes (channel i has 1s at cells belonging to remaining piece i, 0 elsewhere)

Stem: `Conv3d(`$1{+}M$`, 128, `k`=3, `p`=1)` → `BatchNorm3d` → `ReLU`

Body: 6 residual blocks, each:

```
class ResBlock3d(nn.Module):
    def forward(self, x):
        residual = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + residual)
```

Value Head (predicts $P(\text{solvable} \mid \text{state})$):

- *Scale-invariant (GAP):* `Conv3d(128, 64, `k`=1)` → `BN` → `ReLU` → `GlobalAvgPool` → `FC(96, 128)` → `ReLU` → `FC(128, 1)` → `Sigmoid`
- *Legacy (FC):* `Conv3d(128, 32, `k`=1)` → `Flatten` → `FC(`$32N^3$`, 256)` → `FC(256, 1)` → `Sigmoid`

Policy Head (predicts placement logits per voxel):

- *Scale-invariant (Conv):* `Conv3d(128, 64, k=1)` → `BN` → `ReLU` → `Conv3d(96, 1, k=1)` → `Flatten`
- *Legacy (FC):* `Conv3d(128, 64, k=1)` → `Flatten` → `FC(64N³, 512)` → `FC(512, N³)`

Context Features (optional, for scale-invariant heads):

- Fill ratio: fraction of grid occupied
- Remaining volume ratio: total remaining piece volume / grid volume
- Piece count ratio: number remaining / max pieces
- Average piece size: mean volume of remaining pieces / 5
- Projected via `FC(4, 32)` → `ReLU` → `FC(32, 32)`, concatenated into heads

Total parameters: ∼4.6M (128 hidden channels, 6 residual blocks).

5.2 State Encoding

```
def encode_state(grid, remaining_pieces, grid_size, max_pieces):
    state = np.zeros((1 + max_pieces, grid_size, grid_size, grid_size))
    state[0] = grid  # occupancy
    for i, piece in enumerate(remaining_pieces):
        for (x, y, z) in piece:
            state[1 + i, x, y, z] = 1.0
    return state
```

5.3 Training

File: `phase2/train.py`

Data generation: From DLX solutions, we generate partial states by placing pieces sequentially. Each partial state is labeled:

- Positive (label = 1.0): the partial state can be completed to a full solution
- Negative (label = 0.0): sampled from impossible configurations (random piece swaps)

Loss function:

$$\mathcal{L} = \lambda_v \cdot \mathrm{BCE}(\hat{v}, v) + \lambda_p \cdot \mathrm{CE}(\hat{\pi}, \pi)$$

where $\hat{v}$ is the predicted value, v is the solvability label, $\hat{\pi}$ are policy logits, and π is the target placement action. In practice, $\lambda_p = 0$ (value-only training produces better results).

Optimizer: Adam (lr = 10^{-3}, weight decay = 10^{-4}). **Schedule:** Cosine annealing to 0 over training epochs. **Gradient clipping:** Max norm 1.0.

Model	Grid	Epochs	Val Acc.	Solve Rate (bw=8)
`soma_3x3x3`	3^3	50	94.8%	∼55%
`soma_3x3x3_adi1`	3^3	50+15	∼95%	∼70%
`soma_3x3x3_adi2`	3^3	50+30	∼96%	∼80%

Table 2: Trained model checkpoints and performance.

6 NN-Guided Beam Search

File: `phase2/nn_solver.py`

6.1 Algorithm

```
function nn_solve(pieces, grid_size, model, beam_width=64):
    beam <- {initial_state(empty_grid, all_pieces)}
    while beam is not empty and not timed out:
        piece <- select_mrv_piece(beam[0])
        candidates <- []
        for state in beam:
            for placement in valid_placements(piece, state):
                child <- apply(state, piece, placement)
                if child has no remaining pieces:
                    return child.solution
                if not has_isolated_pockets(child):
                    child.score <- model.value(encode(child))
                    candidates.append(child)
        beam <- top_K(candidates, beam_width)
    return None
```

6.2 Key Heuristics

MRV (Minimum Remaining Values): At each depth, select the piece with the fewest valid placements. This minimizes the branching factor and prunes dead branches early—the same heuristic used in Algorithm X.

Pocket Pruning: After placing a piece, check for isolated empty regions (connected components of empty cells unreachable from the grid boundary). If any pocket is too small for any remaining piece, the state is pruned.

Beam Diversity: To prevent beam collapse (all K states converging to similar configurations), reserve a fraction of beam slots for structurally diverse states. Diversity is measured by an 8-octant spatial occupancy signature.

6.3 Frontier Search

If beam search fails, the states visited during search (saved as "frontier roots") can be explored more thoroughly:

- **Frontier DFS:** Bounded depth-first search from the top-scoring frontier states, with a transposition table to avoid revisiting equivalent states.
- **Frontier Complete:** Exhaustive search from frontier roots, using MRV ordering and placement ranking by contact score (prefer placements adjacent to existing pieces).

7 Autodidactic Iteration (ADI)

Reference: Agostinelli et al., "Solving the Rubik's Cube with Deep Reinforcement Learning and Search," *Nature Machine Intelligence*, 2019.

ADI is a self-play improvement loop:

1. **Collect data:** Run beam search on random puzzle instances with a deliberately narrow beam (width 8). Record all visited states.
   - States from successful searches $\rightarrow$ label $= 1.0$ (solvable)
   - States from failed searches $\rightarrow$ label $= 0.0$ (model was overconfident)
2. **Retrain:** Combine new ADI data with original supervised data (prevents catastrophic forgetting). Fine-tune the model with reduced learning rate.
3. **Iterate:** Repeat 2–3 rounds.

The key mechanism: by creating negative examples from the model's own failures, ADI teaches the network to distinguish truly promising states from deceptive dead ends. After 2 rounds on $3 \times 3 \times 3$ data, the narrow-beam solve rate improved from $\sim$55% to $\sim$80%.

8 Hybrid Solver and Size-Gated Routing

File: `hybrid_solver.py`

8.1 `hybrid_solve()` Pipeline

1. Quick volume check (reject if $\sum |P_i| \neq N^3$)
2. Try NN beam search (`timeout_nn` seconds)
3. If NN fails and retry enabled: wider beam, doubled timeout
4. If still fails: frontier DFS from saved beam states
5. If still fails: frontier complete (exhaustive from frontier roots)
6. If still fails: DLX exact solver (`timeout_dlx` seconds)
7. Return solution or `None`

8.2 `solve_size_gated()` Routing

Grid Size	Strategy	Rationale
$N \leq 4$	DLX only (30s)	Fast enough for exhaustive search
$5 \leq N \leq 9$	DLX first (up to 600s), then hybrid	DLX reliable within minutes
$N \geq 10$	Block decomposition	DLX on full grid is too slow

Table 3: Size-gated routing strategy.

9 Block Decomposition for Large Grids

9.1 Strategy

For grid size $N \geq 10$ with a divisor d in range $[5, 9]$:

1. Partition the grid into $(N/d)^3$ non-overlapping $d \times d \times d$ sub-cubes.
2. Group pieces by size. For each sub-cube, determine the target volume (d^3) and sample a multiset of pieces whose sizes sum to d^3.
3. Solve each sub-cube independently via DLX (with per-block timeout).
4. Translate each sub-solution back to global coordinates.

5. If any sub-cube fails, retry with a different random allocation (up to `block_planner_trials` attempts).

9.2 Block Size Selection

```
def _find_block_sizes(grid_size, min_block=5, max_block_vol=343):
    sizes = []
    for b in range(min_block, grid_size):
        if grid_size % b == 0 and b**3 <= max_block_vol:
            sizes.append(b)
    return sizes
```

The `max_block_vol` parameter limits individual block size to keep DLX tractable. The rectangular decomposition (`_split_axis`) now handles most grid sizes, with axis parts constrained to the range $[5, 9]$.

10 Rectangular Sub-Box Decomposition

10.1 Motivation

Many grid sizes have no useful cubic divisors: $N = 11$ (prime), $N = 13$ (prime), $N = 16$ (divisors 2, 4, 8—but $8^3 = 512$ is too large), $N = 17$ (prime), etc.

10.2 Algorithm

Instead of requiring $d \mid N$, split each axis independently into parts in the range $[5, 9]$:

```
def _split_axis(n, min_part=5, max_part=9):
    if n < min_part:
        return None
    if n <= max_part:
        return [n]
    # Try most parts first (smallest sub-problems)
    for k in range(n // min_part, 1, -1):
        base = n // k
        extra = n % k
        parts = [base+1]*extra + [base]*(k-extra)
        if all(min_part <= p <= max_part for p in parts):
            return parts
    return None
```

Examples:

- $11 = 6 + 5 \rightarrow$ sub-boxes of shapes 6×6×6, 6×6×5, 6×5×5, 5×5×5
- $17 = 6 + 6 + 5 \rightarrow$ 27 sub-boxes of various rectangular shapes
- $16 = 6 + 5 + 5 \rightarrow$ avoids the problematic 8×8×8 blocks

10.3 Piece Allocation

Pieces are allocated to sub-boxes via randomized greedy assignment:

1. Compute each sub-box's volume $v = g_x \times g_y \times g_z$.
2. For each sub-box, draw pieces from the available pool until the target volume is reached, matching piece sizes to fill exactly.
3. Solve each sub-box with `dlx_solve_rect(pieces, gx, gy, gz)`.
4. Retry with different random allocations on failure.

10.4 Performance by Decomposition Method

N	Decomposition	Sub-boxes	Max Block Vol	Avg Solve
10	$5+5$	8	125	15s
11	$6+5$	8	216	25s
12	$6+6$	8	216	40s
13	$7+6$	8	343	60s
14	$7+7$	8	343	200s
15	$5+5+5$	27	125	69s
16	$6+5+5$	27	216	84s
17	$6+6+5$	27	216	143s
18	$6+6+6$	27	216	234s

Table 4: Solve time by decomposition strategy.

11 Timeout and Safety Mechanisms

11.1 Wall-Clock Timeout (SIGUSR1 + Threading)

For large-scale tests, a wall-clock timeout prevents individual test cases from running indefinitely. We use `SIGUSR1` (not `SIGALRM`) to avoid conflicts with DLX's internal `SIGALRM`-based timeout:

```
def solve_with_timeout(pieces, N, wall_timeout, dlx_timeout):
    old = signal.signal(signal.SIGUSR1, _raise_timeout)
    timer = threading.Timer(wall_timeout,
                            os.kill, args=(os.getpid(), signal.SIGUSR1))
    timer.start()
    try:
        result = solve_size_gated(pieces, grid_size=N, ...)
    except BaseException:
        result = None  # timeout
    finally:
        timer.cancel()
        signal.signal(signal.SIGUSR1, old)
```

11.2 Per-Block DLX Timeout

Each sub-block in block decomposition has its own `SIGALRM` timeout (default 45s). If a single block exceeds its timeout, the allocation is abandoned and retried with different pieces.

11.3 Multiprocessing Isolation

For grid sizes > 6, DLX is run in a separate process via `multiprocessing` to enable hard timeout via process termination. For grid sizes ≤ 6, DLX runs in-process (avoiding overhead and a known segfault with small grids).

12 Solution Verification

File: `phase1/test_cases.py`

Every solution returned by the solver is independently verified:

```
def verify_solution(solution, grid_size, pieces=None):
    expected_cells = {(x,y,z) for x in range(grid_size)
                      for y in range(grid_size)
                      for z in range(grid_size)}
    covered = set()
    for pidx, cells in solution.items():
        for (x, y, z) in cells:
            assert 0 <= x < grid_size  # bounds
        assert cells.isdisjoint(covered)  # no overlap
        covered.update(cells)
        if pieces is not None:
            placed_norm = normalize(cells)
            orientations = get_orientations(pieces[pidx])
            assert placed_norm in orientations  # shape match
    assert covered == expected_cells  # completeness
    return True
```

13 Test Methodology and Results

13.1 Test Infrastructure

File: `test_thorough_3_14.py`

Tests are organized as:

- **Positive tests (∼70%):** Generate a valid puzzle via constructive decomposition, randomly rotate each piece, solve, and verify.
- **Fault tests (∼30%):** Generate structurally unsolvable puzzles and confirm the solver returns `None`.

13.2 Fault Test Strategies

1. **Oversized piece:** Include a piece with bounding box $> N$ (guaranteed unsolvable).
2. **All rods:** Replace all pieces with $1 \times 1 \times K$ rods.
3. **40% swap:** Replace ∼40% of pieces with random polycubes of the same sizes.
4. **Shape duplication:** Replace all same-size pieces with copies of a single shape.
5. **Fully random:** Generate entirely random polycubes summing to N^3.

13.3 Constructive Test Generation

File: `robust_generator.py`

Valid test instances are generated by constructive decomposition:

1. Start with the full N^3 cube as the remaining region.
2. Select a surface cell (adjacent to the boundary of the remaining region).
3. Enumerate all connected pieces of size 3–5 containing that cell within the remaining region.
4. Randomly select a piece whose removal keeps the remaining region connected.
5. Record the piece, subtract from remaining, repeat until the cube is fully decomposed.

This guarantees solvability by construction and produces unbiased piece distributions (∼45% genuinely 3D shapes).

13.4 Results: $N = 3$ through $N = 14$ (1,080 total tests)

N	**Positive**	**Fault**	**Avg Solve (s)**	**Max Solve (s)**	**Method**
3	70/70	30/30	< 0.1	0.2	DLX exact
4	70/70	30/30	0.3	1.5	DLX exact
5	70/70	30/30	2.1	8.4	DLX exact
6	70/70	30/30	5.8	18.2	DLX exact first
7	49/49	21/21	50.3	68.1	DLX exact first
8	35/35	15/15	42.5	62.8	DLX exact first
9	28/28	12/12	85.4	184.3	DLX exact first
10	28/28	12/12	18.7	45.1	Block (5-cube)
11	28/28	12/12	25.4	62.3	Rect blockwise
12	28/28	12/12	42.1	98.5	Block (6-cube)
13	21/21	9/9	61.8	142.7	Rect blockwise
14	21/21	9/9	140.3	284.7	Block (7-cube)
Total	**518/518**	**222/222**			

Table 5: Thorough test results for $N = 3$ through $N = 14$. 100% solve rate on solvable inputs; zero invalid solutions on fault tests.

13.5 Results: $N = 14$ through $N = 24$

N	Positive	Fault	Avg Solve (s)	Method
14	7/7	3/3	199	blockwise_7cube
15	7/7	3/3	69	blockwise_5cube
16	5/5	2/2	84	rect_blockwise
17	5/5	2/2	143	rect_blockwise
18	5/5	2/2	234	blockwise_6cube
19	3/3	1/1	180	rect_blockwise
20	3/3	1/1	125	blockwise_5cube
21	3/3	1/1	152	rect_blockwise
22	2/2	1/1	210	rect_blockwise
23	2/2	1/1	280	rect_blockwise
24	2/2	1/1	190	blockwise_6cube

Table 6: Large-scale test results. $N \geq 19$ limited in seed count by generator timeouts, not solver failures.

13.6 Combined Results

Across all test runs (two independent seed ranges), the solver achieved **380/380 positive tests passed** (100% solve rate) for $N = 3$ through $N = 24$, with independent verification of every solution. $N = 25$ failed only due to generator timeout, not solver failure. Total test cases including fault tests exceeded 1,800.

14 Visualization

File: `phase1/visualization.py`

14.1 Static 3D Plot

Uses matplotlib's `Axes3D.voxels()` to render the solution as colored voxel blocks, with a 20-color palette for piece identification.

14.2 Animated Assembly

Piece-by-piece animation using `matplotlib.animation.FuncAnimation`. Frame i shows pieces 0 through i placed. Export as GIF via `PillowWriter` or display inline via `to_jshtml()`.

15 Code Structure and Reproduction

15.1 Repository Structure

```
project_root/
+-- phase1/                  # Exact cover solver (DLX)
|   +-- polycube.py          # Piece representation & rotation
|   +-- dlx_solver.py        # Dancing Links data structure
|   +-- solver.py            # Exact cover formulation
```

```
|   +-- visualization.py      # 3D plotting & animation
|   +-- test_cases.py         # Unit tests & verification
|
+-- phase2/                   # Neural network solver
|   +-- nn_model.py           # CuboidNet architecture
|   +-- nn_solver.py          # Beam search, frontier DFS
|   +-- data_generator.py     # Training data generation
|   +-- train.py              # Supervised + ADI training
|   +-- trained_models/       # Saved checkpoints (.pt)
|
+-- hybrid_solver.py          # Size-gated orchestrator
+-- robust_generator.py       # Constructive puzzle generator
+-- fixture_a_plus.py         # A+ grading fixture (150 cases)
+-- grading_harness.py        # Evaluation framework
+-- config.py                 # Hyperparameter configuration
+-- demo.ipynb                # Interactive walkthrough notebook
+-- requirements.txt          # Python dependencies
```

15.2 Reproduction Instructions

Prerequisites: Python 3.9+, Linux/WSL (for `SIGALRM`-based timeouts).

```
# 1. Clone the repository
git clone <repo_url> && cd <project_dir>

# 2. Create virtual environment and install dependencies
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# 3. Run the demo notebook
jupyter notebook demo.ipynb

# 4. Quick solver test (Soma cube)
python3 -c "
from phase1.solver import solve
from phase1.test_cases import SOMA_PIECES, verify_solution
sols = solve(SOMA_PIECES)
print(f'Found {len(sols)} solution(s)')
print(f'Verified: {verify_solution(sols[0], 3)}')
"

# 5. Solve a random 10x10x10 puzzle
python3 -c "
from robust_generator import build_robust_constructive_case
from hybrid_solver import solve_size_gated
pieces = build_robust_constructive_case(10, seed=42)
result = solve_size_gated(pieces, grid_size=10, verbose=True)
print(f'Solved via {result[\"submethod\"]} in {result[\"time\"]:.1f}s')
"
```

15.3 Dependencies

```
torch>=2.0
numpy>=1.24
matplotlib>=3.7
```

Optional: `jupyter` (for demo notebook), `Pillow` (for GIF export).

16 References

1. Knuth, D. E. (2000). "Dancing Links." *Millennium Perspect. Comput. Sci.*, arXiv:cs/0011047.
2. Agostinelli, F., McAleer, S., Shmakov, A., & Baldi, P. (2019). "Solving the Rubik's Cube with Deep Reinforcement Learning and Search." *Nature Machine Intelligence*, 1(8), 356–363.
3. Silver, D., et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search." *Nature*, 529, 484–489.
4. Redelmeier, D. H. (1981). "Counting polyominoes: Yet another attack." *Discrete Mathematics*, 36(2), 191–203.